

STUDENT TASK & PERFORMANCE TRACKER

BYOP – Computer Vision

Name: Aryan Agrawal

Slot: F11+F12

Reg No: 24BAI10221

Date: 18/09/26

2. Introduction

Managing academic tasks and keeping track of student performance can become difficult when all the information is maintained separately. Students usually have many assignments, practicals, deadlines and other academic activities. At the same time, mentors need to keep track of different students and their task completion. Managing all this manually can take more time and also there are chances of missing some information.

To solve this problem, we developed Student Task & Performance Tracker, which is a simple web based application for managing student profiles, academic tasks and performance. The main purpose of this project is to keep the important academic information at one place and make it easier to manage.

The project is divided into three main modules: Student Management, Task Management and Performance Dashboard. In Student Management, users can add, update, search and delete student records. In Task Management, tasks can be assigned to students, deadlines can be stored and task status can be changed between pending and completed. The Performance Dashboard gives a quick view of total students, total tasks, completed tasks, pending tasks and overall completion percentage.

The application is developed using Python and Flask for the backend, SQLite for storing the data and HTML, CSS and JavaScript for the frontend. Chart.js is also used to display the task status in a visual form. Git and GitHub were used for version control and maintaining the project files.

A small Computer Vision feature is also included in the project using OpenCV. It checks the quality of an uploaded image by analysing its brightness and blur level. This feature is kept simple and is mainly added to demonstrate the use of Computer Vision in the application.

The main idea is to create a lightweight academic application that combines web development concepts. Through this project, we learned about database operations, CRUD, APIs, frontend-backend connection and testing.

3. Problem Statement

In a college, students have to manage many academic tasks like assignments, lab work, projects and other activities. Keeping records of students and their tasks manually can become difficult, especially when the number of students and tasks increases. It can also be difficult for mentors to know which tasks are completed and which tasks are still pending.

There is also a problem when student information and task information is kept at different places. It takes more time to check the records and calculate the overall performance of students. A simple system is needed where student details, tasks and completion status can be managed together.

The Student Task & Performance Tracker is developed to solve this problem. It provides a single web application where student profiles can be maintained, tasks can be assigned and their completion can be tracked. The dashboard also gives a quick summary of the academic progress using numbers, percentage and charts.

The project mainly focuses on making academic task tracking simple, organized and easy to understand. It is designed as a lightweight application which can be used by students or academic mentors without requiring a complicated setup.

Objectives

The main objectives of this project are:

- To maintain student academic profiles in one place.
- To add, update, search and delete student records.
- To create and manage academic tasks for students.
- To track whether a task is pending or completed.
- To monitor deadlines of assigned tasks.
- To calculate overall task completion percentage.
- To show student performance in a simple dashboard.
- To reduce manual work in maintaining academic task records.
- To apply web development, database and software testing concepts in a practical project.

4. Functional Requirements

The functional requirements describe what the system should be able to do. The project has three main functional modules: Student Management, Task Management and Performance Dashboard.

4.1 Student Management

The Student Management module is used to maintain the academic details of students. The user can perform the following operations:

- Add a new student with details like name, roll number, course, semester and email.
- View the list of all registered students.
- Search students using name, roll number or course.
- Edit the details of an existing student.
- Delete a student record when it is no longer required.
- Validate important fields such as roll number, email and semester.
- Prevent duplicate student records based on unique information.

4.2 Task Management

This module is used for creating and tracking academic tasks assigned to students.

- Add a new task and assign it to a student.
- Store task title, description and deadline.
- View all available tasks.
- Search tasks using title, student or deadline.
- Filter tasks according to their status.
- Change a task from Pending to Completed.
- Edit task information when required.
- Delete tasks from the system.
- Display the current status of each task using status badges.

- Computer Vision: The system can check uploaded images for basic brightness and blur quality using OpenCV.

4.3 Performance Dashboard

The Performance Dashboard provides a quick summary of the academic task data.

- Display the total number of students.
- Display the total number of assigned tasks.
- Show the number of completed tasks.
- Show the number of pending tasks.
- Calculate the overall task completion percentage.
- Display completed and pending tasks using a chart.
- Show individual student task performance.
- Display the completion rate for each student.
- Allow the dashboard data to be refreshed.

4.4 Input and Output

The system takes information such as student details, task details, deadlines and task status as input. This information is stored in the SQLite database and is then used by the application to show student records, task lists and performance statistics.

The main output of the system is the student directory, task management table and performance dashboard.

5. Non-Functional Requirements

Non-functional requirements describe how the system should work, rather than only what features it provides. For this project, we focused on keeping the application simple, reliable and easy to use.

5.1 Usability

The application should be easy to understand for students and academic mentors. The navigation is kept simple with separate pages for Students, Tasks and Dashboard. The buttons, tables and status labels are also designed in a clear way.

5.2 Performance

The application should respond quickly for normal academic usage. Since SQLite is used as the database and the project is lightweight, the basic operations like adding students, creating tasks and loading dashboard data can be performed without much delay.

5.3 Reliability

The system should give correct results when student and task information is entered. Database operations are handled properly and the dashboard calculates the task statistics from the stored records.

5.4 Error Handling and Validation

The system should not accept invalid or incomplete information. Input validation is used for fields such as student details, email, semester and task information. Duplicate records and invalid inputs are also handled with appropriate responses.

5.5 Maintainability

The project is divided into different files and folders instead of keeping all the code in one file. Routes, frontend scripts, templates, database logic and tests are separated. This makes it easier to understand and modify the project later.

5.6 Resource Efficiency

The application uses a lightweight technology stack consisting of Flask, SQLite, HTML, CSS and JavaScript. It does not require a large database server or unnecessary external services, which keeps the resource requirement relatively low.

5.7 Security

Basic validation is included to avoid incorrect or unwanted data being stored in the database. The application is mainly designed as an academic project, so advanced authentication and role-based security are outside the current scope

6. System Architecture

The Student Task & Performance Tracker follows a simple three-layer web application architecture. The main layers are the Frontend, Backend and Database. This structure helps in keeping the different parts of the project separated and easier to manage.

6.1 Frontend Layer

The frontend is the part which the user interacts with. It is developed using HTML, CSS and JavaScript.

The frontend contains the following main pages:

- Home Page – provides an overview of the application and navigation to the main modules.
- Student Management – used for managing student records.
- Task Management – used for creating and tracking tasks.
- Performance Dashboard – displays task statistics and student performance using charts.

JavaScript is used to send requests to the backend and update the page according to the returned data.

6.2 Backend Layer

The backend is developed using Python Flask. It handles the application logic and connects the frontend with the database.

The backend mainly contains:

- Flask application configuration
- Route handling
- API endpoints
- Student CRUD operations
- Task CRUD operations
- Task status management
- Dashboard statistics calculation

- Input validation and error handling
- Computer Vision image quality analysis using OpenCV
- The application uses API routes for performing operations on students and tasks. The dashboard uses the `/api/dashboard/stats` endpoint to get the required performance statistics.

6.3 Database Layer

SQLite is used as the database for storing the application data.

The database mainly stores:

- Student information
- Task information
- Relationship between students and their assigned tasks
- Task status and deadlines

SQLite was selected because this is a lightweight academic project and it does not require a separate database server.

6.4 Overall Data Flow

The basic working flow of the system is:

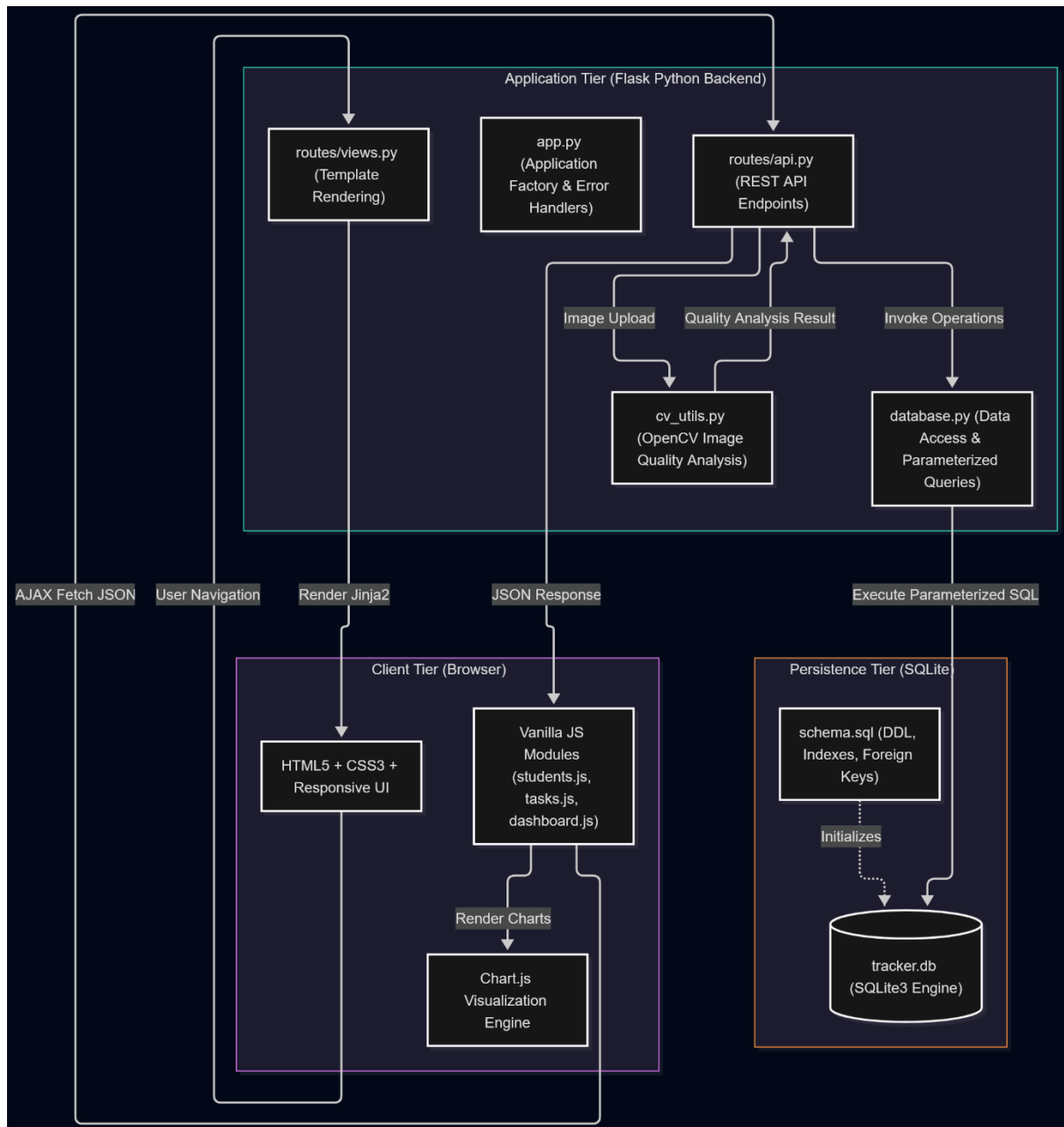
User → Frontend → Flask Backend → SQLite Database

For the Computer Vision feature, uploaded images are sent to the Flask API and processed by the OpenCV module before the quality result is returned to the user.

For example, when a user adds a student, the information is entered through the frontend. The request is sent to the Flask backend, where it is validated and stored in the SQLite database. The updated information is then returned and displayed on the frontend.

For the dashboard, the backend reads the task information from the database, calculates the required statistics and sends the results to the frontend. JavaScript then displays these results in the dashboard and chart.

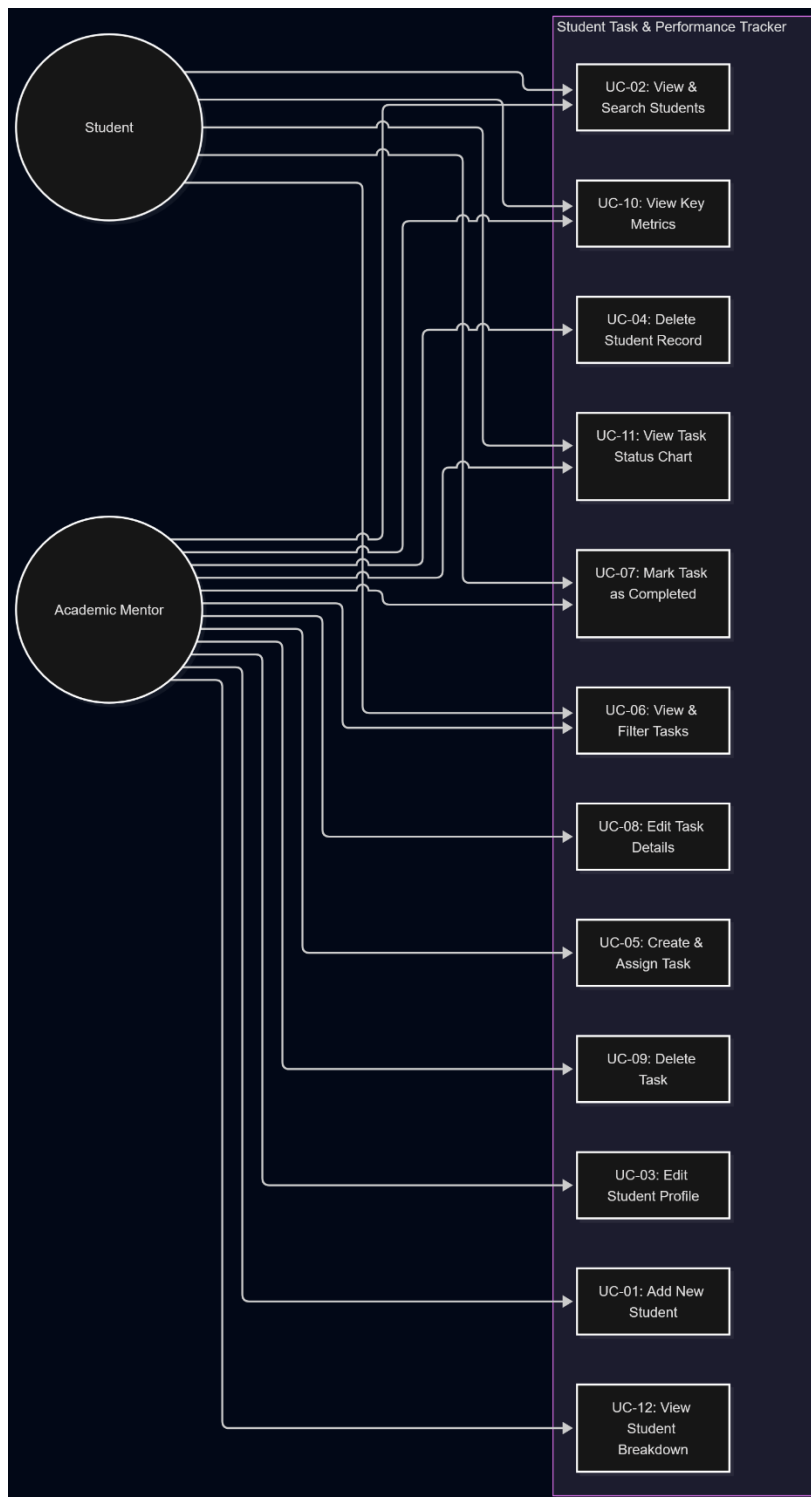
6.5 Architecture Diagram



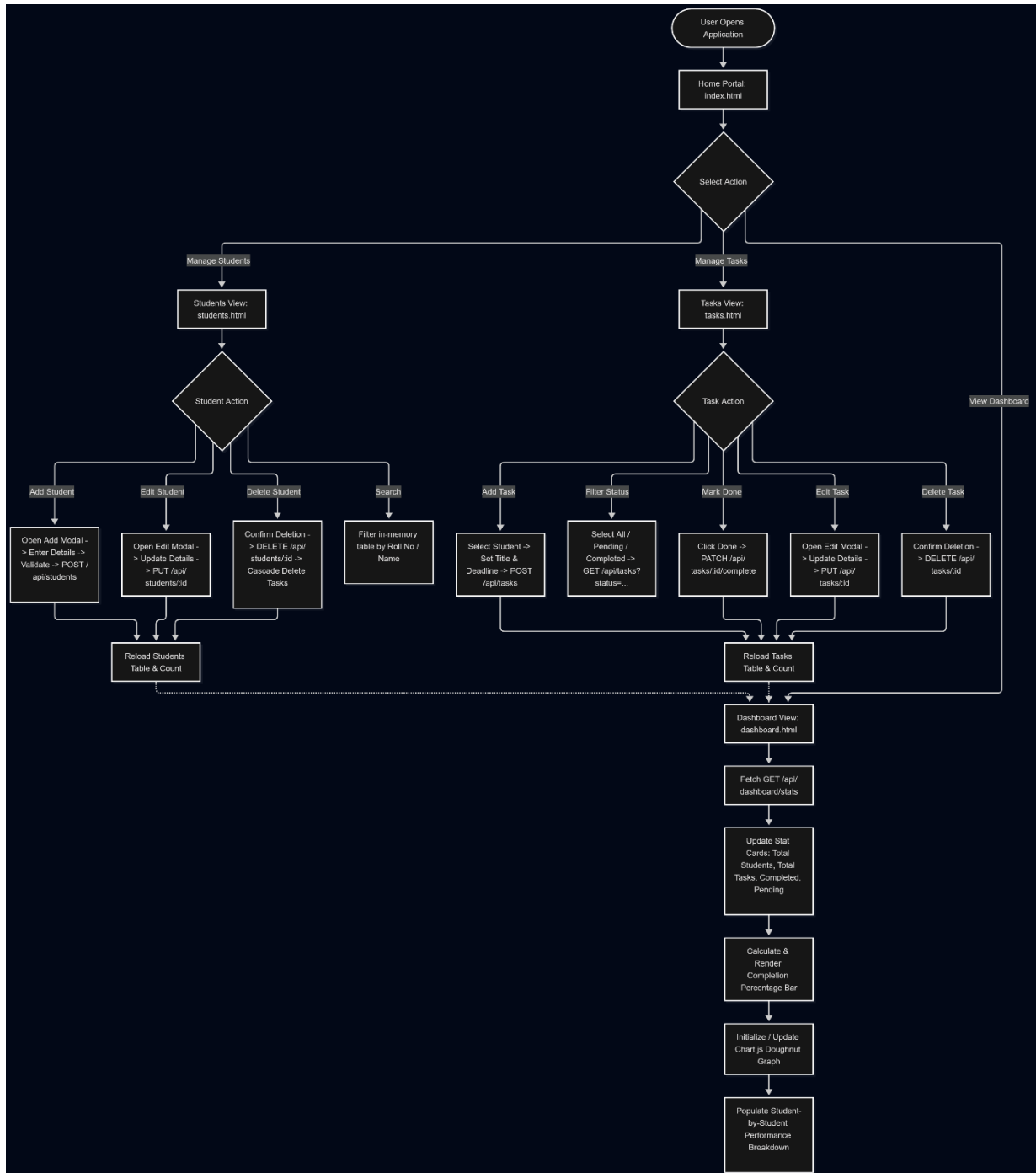
7. Design Diagrams

The project follows a modular architecture where responsibilities are distributed across different routes, modules, components and database functions.

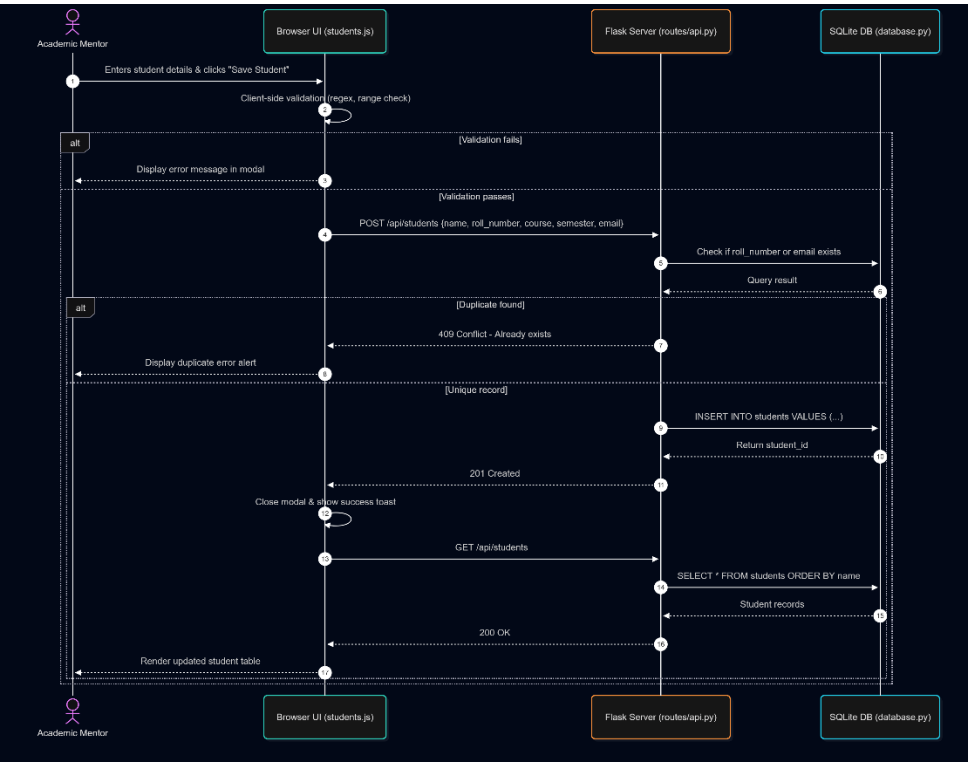
7.1 Use Case Diagram



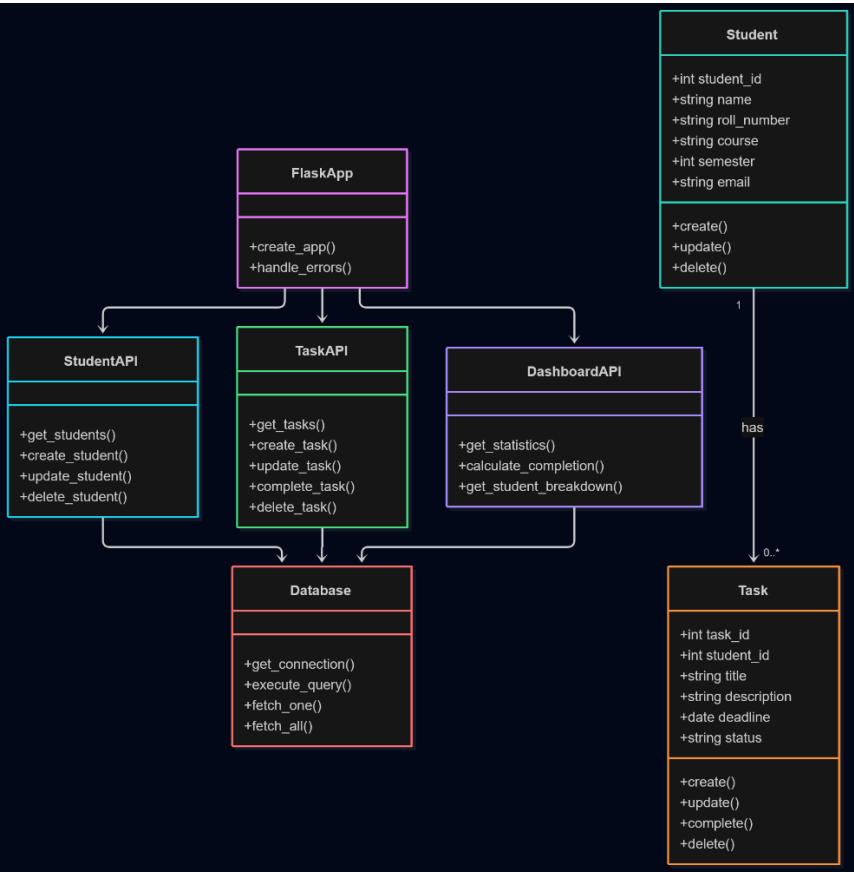
7.2 Workflow Diagram



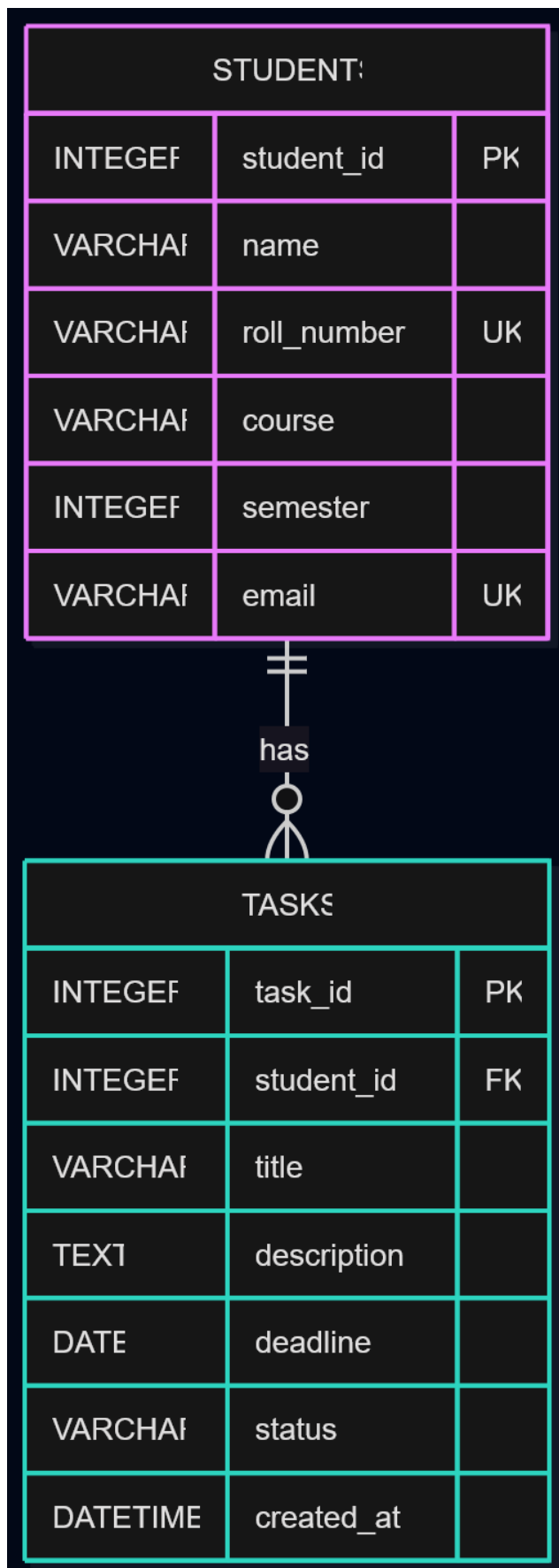
7.3 Sequence Diagram



7.4 Class Diagram



7.5 ER Diagram



8. Design Decisions & Rationale

During the development of Student Task & Performance Tracker, some design decisions were taken to keep the project simple, easy to understand and also properly structured.

1. Flask for Backend:

Flask was selected because it is lightweight and easy to use for a small web application. It also helped us create API endpoints and connect the frontend with the database.

2. SQLite for Database:

SQLite was used because the project does not require a large database server. It is simple to setup and stores the complete database in one file. It also supports tables, foreign keys and SQL queries.

3. HTML, CSS and Vanilla JavaScript:

We used basic frontend technologies instead of a large frontend framework. This made the application easier to develop and understand. JavaScript is used for API requests, validation, table updates and dashboard interactions.

4. Three Main Modules:

The system was divided into Student Management, Task Management and Performance Dashboard. This separation makes the project more organized and each module has its own specific purpose.

5. REST API Communication:

REST API endpoints were used for communication between frontend and backend. Different HTTP methods such as GET, POST, PUT, PATCH and DELETE are used according to the operation.

6. Chart.js for Visualization:

Chart.js was selected to display completed and pending tasks in a doughnut chart. This makes the performance information easier to understand compared to showing only numbers.

7. Automated Testing:

Pytest was used for testing important backend operations. It helped us check student operations, task operations, validation and dashboard calculations before final submission.

Overall, these decisions were made to keep the application lightweight while still demonstrating important concepts like CRUD operations, database management, API communication, modular design and software testing.

9. Implementation Details

The Student Task & Performance Tracker was implemented as a web based application using Python, Flask, SQLite, HTML, CSS and JavaScript. The application was divided into different files and folders so that each part of the system can be managed separately.

1. Backend Implementation:

Flask is used as the main backend framework. The app.py file starts the application and connects the different routes. The routes/views.py file handles the web pages, while routes/api.py contains the REST API endpoints for student, task and dashboard operations.

2. Student Management:

The Student Management module provides CRUD operations for student records. Users can add student details such as name, roll number, course, semester and email. The system validates the entered information and also checks for duplicate roll numbers and email addresses.

3. Task Management:

The Task Management module is used to create and manage academic tasks. Each task is connected with a student using student_id. Users can add task title, description, deadline and status. Tasks can be edited, deleted and marked as completed.

4. Performance Dashboard:

The dashboard collects information from the database and calculates total students, total tasks, completed tasks, pending tasks and completion percentage. The data is shown using summary cards, a progress bar, a Chart.js doughnut chart and student-wise performance information.

5. Database Implementation:

SQLite is used for storing application data. The main tables are students and tasks. A foreign key relationship connects tasks with students. SQL queries are handled through database.py, where parameterized queries are used for database operations.

6. Frontend Implementation:

HTML is used for the page structure and CSS is used for the user interface. JavaScript files such as students.js, tasks.js and dashboard.js handle frontend operations and communicate with the Flask APIs using fetch requests.

7. Testing Implementation:

Pytest was used to test important application functions. The tests cover student operations, task operations, validation, duplicate records and dashboard calculations. The final automated test run passed all 23 tests successfully.

8. Computer Vision Implementation:

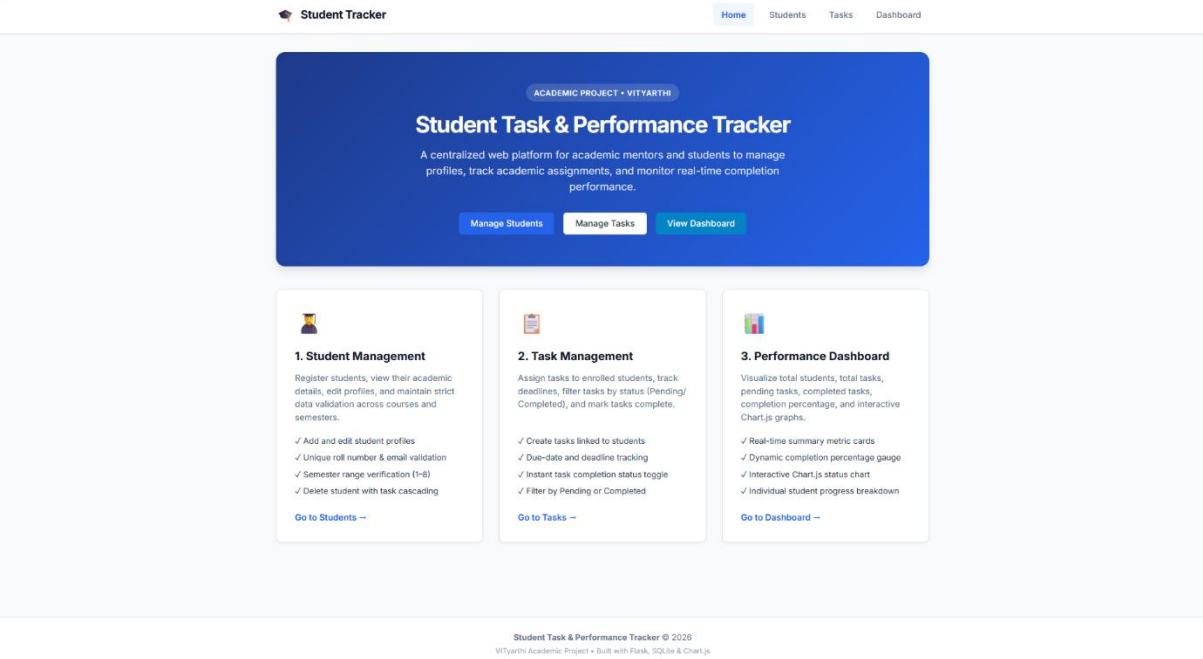
OpenCV is used to analyse uploaded images. It checks the image brightness and blur level using grayscale intensity and Laplacian variance. The image is processed in memory and is not stored in the database.

9. Project Organization:

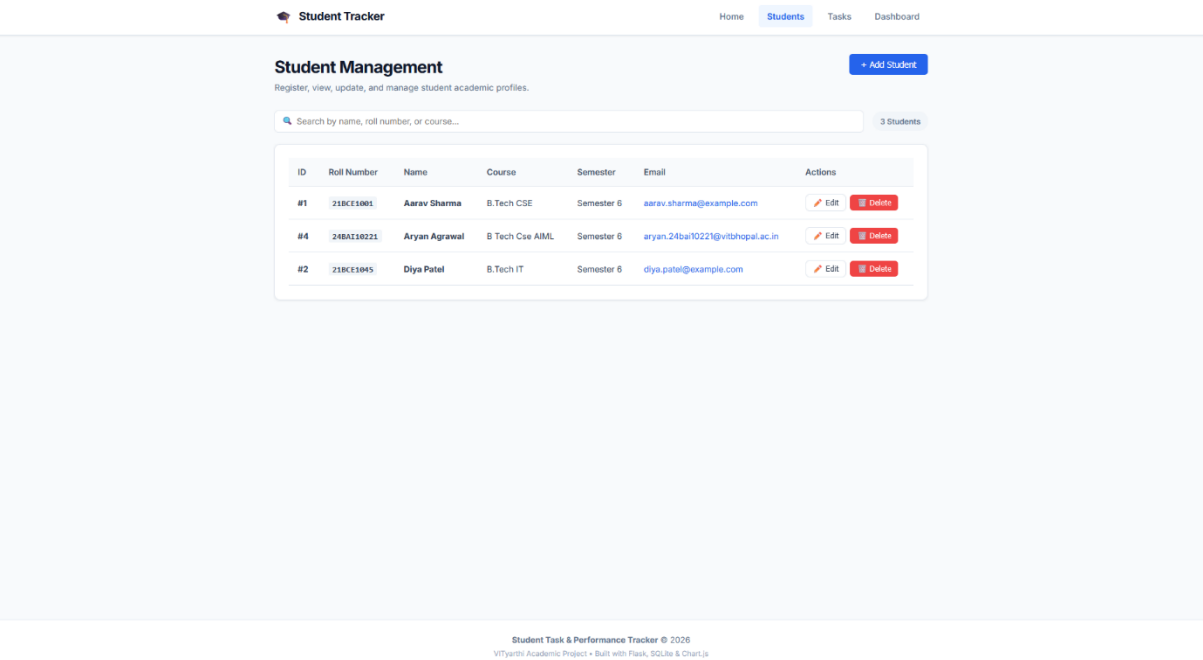
The project source code is separated into routes, templates, static files, tests and documentation folders. This structure makes the project easier to understand, test and maintain.

10. Screenshots / Results

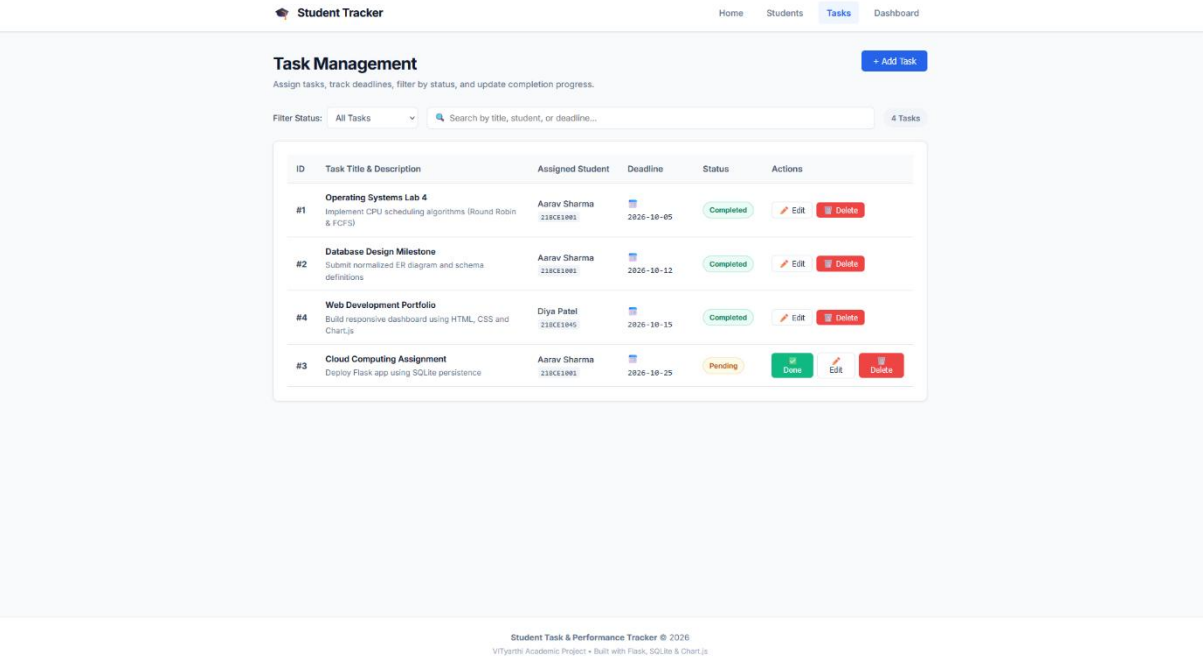
10.1 Home Portal



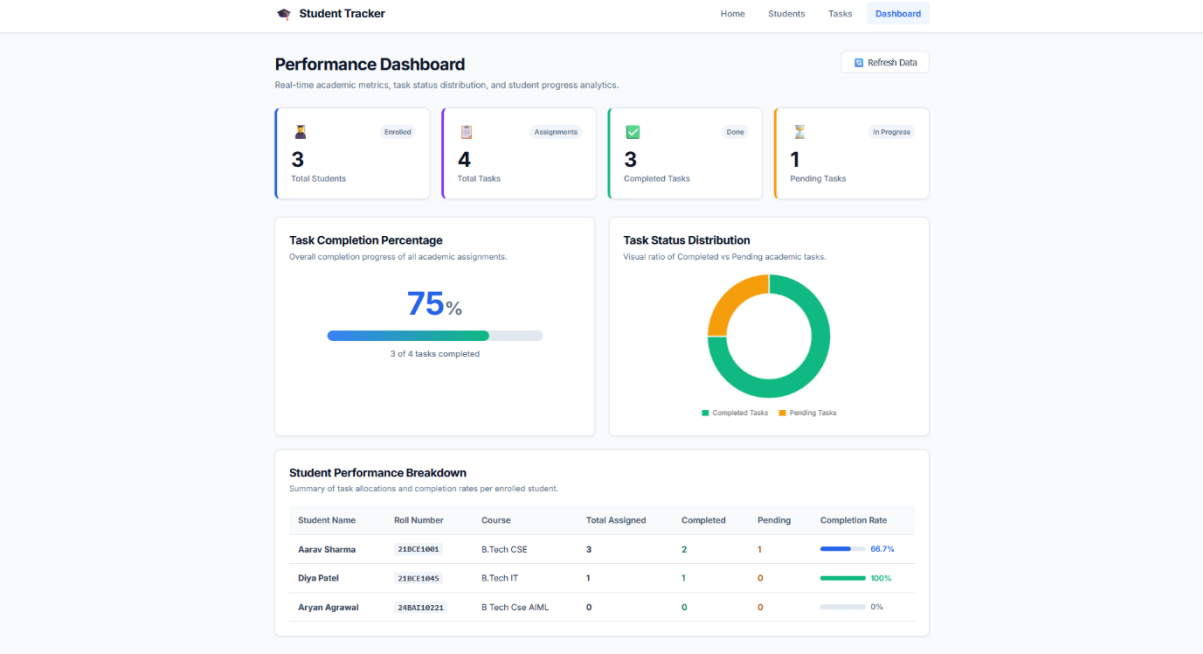
10.2 Student Management



10.3 Task Management



10.4 Performance Dashboard



11. Testing Approach

Testing was done to make sure that the main functions of the Student Task & Performance Tracker are working correctly. Both automated testing and manual testing were used during the development of the project.

1. Automated Testing:

Pytest was used for testing the backend functionality. Different test cases were created for student management, task management and dashboard operations.

2. Student Management Testing:

The student module was tested for adding, viewing, updating and deleting students. Validation was also tested for invalid inputs and duplicate roll numbers or email addresses.

3. Task Management Testing:

The task module was tested for creating, editing, deleting and completing tasks. Task filtering for pending and completed status was also checked.

4. Dashboard Testing:

Dashboard calculations were tested with different database conditions. The system was also tested when there are no students or tasks to make sure that the application does not produce calculation errors.

5. Manual Testing:

The application was opened in the browser and the main pages were manually checked. Student records were added and modified, tasks were created and completed, and the dashboard was checked after making changes.

6. Final Test Result:

The final automated test run passed all 23 tests successfully. This confirmed that the tested student, task and dashboard functionalities were working as expected.

Overall, testing helped us find errors during development and verify that the important functions of the application were working properly.

12. Challenges Faced

During the development of the Student Task & Performance Tracker, we faced some challenges while connecting different parts of the application. These problems also helped us understand the practical side of web application development.

1. Connecting Frontend with Backend:

One of the initial challenges was connecting the JavaScript frontend with the Flask backend. We had to understand how API requests and JSON responses work between the browser and server.

2. Database Operations:

Managing the SQLite database was another challenge. We had to properly create tables, use foreign keys and write SQL queries for adding, updating, deleting and retrieving records.

3. Input Validation:

We needed to make sure that incorrect student and task information was not stored in the database. Validation was added for required fields, email format, semester range and duplicate records.

4. Dashboard Calculations:

Calculating completed and pending tasks and the overall completion percentage required proper SQL queries and handling of cases where there are no tasks in the database.

5. Maintaining Project Structure:

As the project became bigger, keeping the files organized was also important. Separating routes, templates, JavaScript files, tests and documentation helped us manage the project better.

6. Testing and Debugging:

Some issues were found while testing the application. We had to check API responses, database operations and frontend behaviour to identify and fix these problems.

7. Git and GitHub:

Managing the project using Git and GitHub was also a learning experience. We learned how to commit changes, push the project and maintain the project files in a proper repository.

These challenges were useful because they gave us practical experience in developing, testing and maintaining a complete web based application.

13. Learnings & Key Takeaways

1. Web Development:

We learned how frontend technologies like HTML, CSS and JavaScript can be connected with a Python Flask backend to create a working web application.

2. Backend and API Development:

We understood how REST API endpoints work and how GET, POST, PUT, PATCH and DELETE requests are used for different operations.

3. Database Management:

We learned how to create tables, use primary keys and foreign keys, and perform CRUD operations using SQL with SQLite.

4. Modular Programming:

Dividing the project into different routes, JavaScript files, templates and database functions helped us understand why proper project structure is important.

5. Data Visualization:

We learned how dashboard data can be converted into useful visual information using charts and completion percentages.

6. Software Testing:

Using Pytest helped us understand how automated test cases can be used to check whether different functions are working correctly.

7. Git and GitHub:

We also learned how to use Git for version control and GitHub for maintaining and sharing the project source code.

8. Problem Solving:

Debugging API errors, database issues and frontend problems improved our ability to identify problems and find suitable solutions.

Overall, this project gave us practical experience in web development, database management, API integration, testing and version control. It also helped us understand how a complete software project is planned, developed and tested.

14. Future Enhancements

The current version of the Student Task & Performance Tracker covers the main requirements of student, task and performance management. However, some additional features can be added in the future to make the system more useful and complete.

1. User Login and Authentication:

A login system can be added for students and academic mentors. Different users can have different access permissions according to their role.

2. Notifications and Reminders:

The system can provide notifications for upcoming deadlines and pending tasks. Email or browser notifications can also be added.

3. Advanced Performance Analytics:

More performance metrics can be added, such as weekly or monthly progress, overdue tasks and individual student performance trends.

4. Mobile Responsive Improvements:

The interface can be improved further for mobile devices so students can easily use the system from their phones.

5. Search and Filtering:

More advanced filters can be added for courses, semesters, deadlines and student performance.

6. Cloud Database:

SQLite can be replaced with a cloud based database when the number of users and records becomes larger. This can allow data to be accessed from different locations.

7. AI Based Suggestions:

In the future, AI can be used to identify students who are falling behind and suggest better task schedules based on their pending work and completion history.

8. Deployment:

The application can be deployed on a cloud platform so that it can be accessed online instead of only running locally.

These enhancements can make the application more scalable and useful for real academic environments.

15. References

1. Flask Documentation - Python Web Framework documentation and API development concepts.
2. Python Documentation - Python programming language reference and standard library.
3. SQLite Documentation - SQLite database, SQL queries and database concepts.
4. MDN Web Docs - HTML, CSS and JavaScript concepts used for frontend development.
5. Chart.js Documentation - Documentation for creating charts and data visualizations.
6. Pytest Documentation - Documentation for Python testing and automated test cases.
7. Git Documentation - Git version control concepts and commands.
8. GitHub Documentation - Repository management and version control using GitHub.
9. VITyarthi Build Your Own Project Guidelines - Project requirements, documentation and evaluation guidelines provided for the course project.